

JavaScript Day One

- Installed VS code on ubuntu through terminal. Installed various libraries such as ES linter and prettify.
- Learned how you need html to connect to browser (pressing shift + !) makes boilerplate code for html.
- In the body of the boilerplate we connect our Javascript with html through `<script src = "script.js"></script>`.
- Variables are dynamic, no need for specifying type can be changed later as well.
- `fullName = "Saboor Ahmed";`
- Scope: let, var and const. Mostly let and const are used. `let fullName = "sab";`
- Before 2015 only var existed and then we got ES6(ECMAScript 6). Var could be re-declared as well as re-updated. let can not be re declared but can re update. const cannot be changed they are constants. let can be declared (undefined) but not initialized and this does not work for const as const should be initialized.
- Var is a global scope variable, however let and const are block scope variables. Block is defined by curly brackets such as `{ code }` is a block.
- Primitive data types (7) & Non primitive data types (eg object: to be discussed later).
- Primitive data types = Number, string, boolean, undefined (no values), null (special values which is nothing), BigInt, Symbol. `let x = BigInt("123");`
- Non primitive data types (object (collection of values), arrays, functions) to be discussed later.
- Example of object

```
const student = {  
  fullName: "Saboor Ahmed",  
  age: 23  
};  
  
console.log(student.age);
```

- Or `console.log(student["age"]);`
- We can update the values of the key in the object later like `(student.age = 25;)`. So we can update const object key.
- `typeof fullName;` if individual variable. `typeof student["fullName"];` if object.
- Comment: `//` single line, `/* */` muti line comment.
- Operators are arithmetic operators such as `(+, -, *, /)`.

```
let a = 5;  
let b = 2;  
•
```

- `console.log("a + b = ", a+b);`

- Moreover we have Modulus that is the remainder of a divided by b. For example $5 \% 2$ is equal to 1 that is the remainder. Exponentiation: $a^{**}b$ is equal to a to the power of b that is $2^{**}3$ equals $2*2*2$.

- Unary operators increment and decrement is `a++`; and `b--`;

- `a++` -> post increment, `--a` -> pre increment. Post increment value change after running this line. Pre increment value changes first. For example

```
let a = 5;
```

```
console.log(a++); //5
```

And

```
console.log(++a); //6
```

- Assignment operators(`=`, `+=`, `-=`, `/=`, `*=`, `%=`, `**=`)
- Comparison operator(`==`, `===`, `!=`, `!==`, `>`, `>=`, `<`, `<=`) “`===`” check value and data type.
- Logical operators(`&&`, `||`, `!`)
- Conditional statements: if statement

```
let age = 16;

if (age > 18) {
  console.log("You can vote");
}

if (age < 18) {
  console.log("You can not vote");
}
```

- If-else statement:

```
let mode = "white";

if (mode === "white") {
  console.log("You are not in dark mode");
} else {
  console.log("You are in dark mode");
}
```

- Else if statement:

```
let mode = "white";

if (mode === "white") {
  console.log("You are not in dark mode");
} else {
  console.log("You are in dark mode");
}
```

```
}
```

- Ternary Operator

Condition? True output : false output

```
let age = 30;

let result = age >= 18 ? "adult" : "not adult";

console.log(result);
```

- Switch Statement:

```
const expr = "papayas"

switch(expr) {
  case("oranges"):
    console.log("oranges");
    break;
  case("papayas"):
    console.log("papayas");
    break;
  default:
    console.log("we don't have it");
}
```

- For loop:

```
let sum = 0;

for (let i = 1; i <= 5; i++) {
  sum = sum + i;
}

console.log(sum);
```

The 'i' defined in for loop is inside the block scope of that for loop and not accessible outside.

- While loop:

```
let i = 1;
while(i <= 5){
  console.log("i: ", i);
  i++;
}
```

- Do-while loop

```
let i = 1;
```

```
do {
  console.log("Hello World");
  i++;
} while (i < 5);
```

- For of loop: used for strings and arrays

```
let str = "Hello";
for(let i of str){
  console.log(i);
}
```

For in loop: used for objects and strings, return the keys of objects.

- Strings are immutable.
- String methods:
 - str.length;
 - str[0],
 - str.toUpperCase(),
 - str.toLowerCase(),
 - str.trim() (remove whitespaces);
 - str.slice(start,end(not included)) // returns part of string
 - str.concat(str2) //joins str2 with str
 - str.replace(oldvalue, newvalue) // only first matching replace else use replaceAll
 - str.charAt(idx)
- Template Literal for an output with variables. Example

```
let n = "saboor";
let output = `Name is ${n}`;
console.log(output);
```

Name is saboor

This is also called string interpolation eg let output = `output of 2+5 = \${2+5}`;

- Escape character: /n (newline), /tab(give space of 5), they are counted as one character.
- Arrays: let arr = [10,20,30];

```
let arr = [10, 20, 30];
for(let el of arr){
  console.log(el)
}
```

arr.push(50) adds at end, arr.pop() removes from end. arr.toString() converts array to string.

arr.concat(arr2);

arr.unshift(5) adds value at start, arr.shift() removes value from start.

arr.slice(startidx, endidx) same as str.slice (does not change original).
arr.splice(startidx, delcount, newelements) does change original array. Splice can add, delete or replace elements in array.

- Functions

```
function sum(x,y)
{
    return x+y;
}

let s = sum(5,12);
console.log(s);
```

Arrow Functions

```
const arrowSum = (a,b) => {
    return a+b;
};

console.log(arrowSum(2,3));
```

- Function associated with an object becomes method."For each" is a method for arrays as it is a function associated with object array it becomes a method.
- Functions in javascripts can be passed as parameters to another function similarly, they can be returned as well.
- A callback is a function passed as a function to another function.
- forEach loop: this is a method as well.

```
let arr = [1,2,3,4,5];

arr.forEach(function myFun(val) {
    console.log(val);
});
```

Or

```
let arr = [1,2,3,4,5];

arr.forEach( (val) => {
    console.log(val);
});
```

The parameters can be (value, index, arr)

- Higher order function is a function that contains function as a parameter or can return a function for example forEach is a higher order function/method.

- `arr.map()` returns a new array but same as `forEach`

```
let arr = [1,2,3,4,5];

let newArr = arr.map( (val) => {
  return val;
});

console.log(newArr);
```

- `arr.filter()` filters out specific value from the array.

```
let arr = [1,2,3,4,5];

let newArr = arr.filter( (val) => {
  return val % 2 === 0;
})

console.log(newArr);
```

- `arr.reduce()` reduces to any one value for example sum or avg or factorial.

```
let arr = [1,2,3,4,5];

let result = arr.reduce( (res, val) => {
  return res + val;
});

console.log(result);
```

`res` is at index 0 and `val` is at index 1. Eventually results store the returned value.
Find largest in an array.

```
let arr = [1,2,3,4,5];

let result = arr.reduce( (prev, curr) => {
  return prev > curr? prev: curr;
});

console.log(result);
```